

AP COMPUTER SCIENCE PRINCIPLES · THE UNIVERSAL STUDY GUIDE

Targeting a 3, 4, or 5 — every concept tagged for your tier

Tier badges: [3] foundational · [4] adds for 4+ · [5] adds for 5

Targeting a 4 → study [3] AND [4] items. Targeting a 5 → study all three tiers.

PAGE 1 | MATH OF 3/4/5 · THE TWO-PART EXAM · PSEUDOCODE

About this exam. AP CSP has TWO components: (1) the **Create Performance Task** — a program you build and submit before exam day (worth ~25%), and (2) the **End-of-Course Exam** — 70 multiple-choice questions in 2 hours (~75%). Unlike most APs, **half your battle is already done if you handled the Create PT well**. The exam itself uses **AP-style pseudocode**, not Python or Java.

THE MATH OF YOUR TARGET — combined score breakdown

Target	MCQ raw (70 q, ~75%)	Create PT (~25%)	Strategy
3 (PASS)	~33-38 / 70 (~50%)	~3/6 row-points	[3] only — focus pseudocode literacy + simple PT
4	~42-46 / 70 (~63%)	~4-5/6	[3] + [4] — must understand iteration, lists, abstraction
5	~52-58 / 70 (~78%)	~5-6/6	All — including Big Idea 4 (networks) details + cybersecurity

Critical: The PT is graded on **6 specific row-points** (program purpose, function development, list, iteration, parameters, return values). Hitting all 6 is essentially required for a 5.

THE 5 BIG IDEAS — exam weight breakdown

#	Big Idea	Weight	Core focus
1	Creative Development	10–13%	Program design, debugging, collaboration, iterative process
2	Data	17–22%	Binary, abstraction, compression (lossy/lossless), metadata
3	Algorithms & Programming	30–35%	The biggest unit. Variables, lists, conditionals, loops, procedures, abstraction
4	Computer Systems & Networks	11–15%	Internet, packets, fault-tolerance, parallel/distributed, redundancy
5	Impact of Computing	21–26%	Surprisingly heavy. Digital divide, bias, crowdsourcing, IP, cybersecurity, PII

KEY INSIGHT Big Idea 5 is **sneaky**. Students underestimate it because it sounds soft. **It's 21-26% of the exam — bigger than Big Ideas 1, 2, and 4.** Don't just learn coding — learn ethics, bias, digital divide, IP. These are easy points if you read carefully.

AP-STYLE PSEUDOCODE — your only language [3]/[4]

AP CSP doesn't use Python, Java, or JavaScript on the exam. Everything is in **AP pseudocode** (similar to App Inventor / Snap blocks rendered in text). Memorize these forms:

```

ASSIGNMENT:    a ← expression      (a gets the value of expression)
COMPARISON:    a = b   a ≠ b   a < b   etc.
IF/ELSE:       IF (condition) { block } ELSE { block }
REPEAT N:      REPEAT n TIMES { block }
REPEAT UNTIL:  REPEAT UNTIL (condition) { block }
FOR EACH:      FOR EACH item IN list { block }
PROCEDURE:     PROCEDURE name(parameters) { block; RETURN(value) }
LIST:          aList ← [10, 20, 30]  aList[1] is FIRST item (1-indexed!)
LIST OPS:      INSERT(aList, i, value) APPEND(aList, value) REMOVE(aList, i)
               LENGTH(aList)         aList[i] ← value
ROBOT (rare):  MOVE_FORWARD()  ROTATE_LEFT()  ROTATE_RIGHT()  CAN_MOVE(dir)
  
```

WATCH OUT **LISTS ARE 1-INDEXED** in AP pseudocode. `aList[1]` is the FIRST item, NOT the second. This is the #1 source of MCQ mistakes.

PAGE 2 | BIG IDEA 1: CREATIVE DEVELOPMENT (10–13%)

Program Design Process [3]

Phase	What happens
Investigating	Identify needs, gather requirements, study users, research.
Designing	Plan structure, draw flowcharts, decide data structures, write pseudocode.
Prototyping	Build a minimum version. Test ideas before fully committing.
Testing	Run with diverse inputs, find bugs, gather user feedback.
Iterating	Improve based on feedback. Repeat — development is iterative, not linear .
KEY INSIGHT [3] On the MCQ, look for "iterative" or "incremental" answers when a question asks how programs are typically developed. Linear/waterfall answers are wrong.	

Errors & Debugging [3]/[4]

Error type	What it means
Syntax error [3]	Code doesn't follow language rules. Found by compiler/interpreter — won't run at all.
Logic error [3]	Code runs but produces WRONG output. Found by testing with known inputs.
Runtime error [3]	Code starts running, then crashes (e.g., divide by zero, index out of range).
Overflow error [4]	Number too big for the storage allocated → wraps around or crashes.
Round-off error [4]	Decimal stored imprecisely (e.g., $0.1 + 0.2 \neq 0.3$ exactly in binary).

Debugging strategies [3]/[4]: print/display intermediate values, hand-trace step by step, use a debugger, comment out sections, check edge cases (empty list, zero, negative, max value).

Program Documentation [3]

- **Comments [3]** = notes in code (ignored by computer) explaining what code does or why.
- **Documentation [3]** should include: purpose, inputs/outputs, data abstractions, procedure descriptions, acknowledgments of code from elsewhere.
- **[4] Why document?** Other programmers (and YOU in 6 months) need to understand the code. Reduces bugs during maintenance.

Collaboration [3]/[4]

- **Pair programming [3]:** 2 people, 1 computer — "driver" types, "navigator" reviews. Catches bugs immediately.
- **[4] Diverse perspectives** improve programs — different users have different needs. Without diversity in development, software has bias and accessibility blind spots.
- **[4] Crowdsourcing** = getting input/data/contributions from a large group online. Examples: Wikipedia, OpenStreetMap, citizen science.

PAGE 3 | BIG IDEA 2: DATA — BINARY · ABSTRACTION · COMPRESSION (17–22%)**Binary & Number Systems [3]/[4]**

BINARY (base 2): only 0 and 1. Each digit = 'bit'. 8 bits = 1 byte

Place values: 128 64 32 16 8 4 2 1
 1 0 1 1 0 0 1 0 = 128+32+16+2 = 178

N bits → 2^N possible values. 8 bits → 256 values (0–255)

HEX (base 16): 0–9, A–F. 1 hex digit = 4 bits. #FF = 11111111 = 255

Color hex #RRGGBB: 6 hex digits = 24 bits = ~16.7 million colors

Abstraction in Data [3]/[4]/[5]

- **Abstraction [3]** = hiding details to focus on essentials. **Letters in computers** are abstractions over binary (ASCII/Unicode codes).
- **[4] Layers of abstraction:** physical electricity → bits → bytes → characters → strings → words → documents. Each layer hides the layer below.
- **[5] Why abstraction matters:** lets you build complex systems without re-inventing each layer. A web developer doesn't think about transistors.

Compression — lossy vs lossless [3]/[4]

Type	Lossless	Lossy
Definition	Original data fully reconstructable from compressed form.	Some data permanently discarded — cannot recover original.
Example formats	ZIP, PNG, FLAC, GIF, RAW (text)	MP3, JPEG, MP4 (video), most streaming
When to use [4]	Need exact original — text, code, medical scans, archives.	Acceptable to lose detail — photos, audio, video. Smaller files.
Tradeoff	Larger files, no quality loss.	Much smaller files, but quality degrades. Cannot "undo".

KEY INSIGHT [4] Common MCQ trap. A photo saved as JPEG, then JPEG'd again, loses MORE quality each time. Lossy compression is **not reversible**. The original detail is **gone forever**, not hidden.

Data, Information, Knowledge [3]/[4]

- **Data [3]** = raw values. "72" by itself.
- **Information [3]** = data with context. "72°F at noon Tuesday".
- **Knowledge/Insight [4]** = patterns from information. "Tuesdays are warmer than Mondays in March."
- **Metadata [4]** = data ABOUT data. A photo's metadata: date taken, camera, GPS coordinates, file size.

Cleaning & Filtering Data [4]

- **[4] Data cleaning** = fixing/removing incorrect, incomplete, duplicate, or wrongly-formatted data BEFORE analysis.
- **[4] Filtering** = removing data that doesn't meet a criterion (e.g., keep only sales > \$100).
- **[4] Bias in data** = if your data sample doesn't represent the real population, your conclusions will be wrong (Big Idea 5 connection).

PAGE 4 | BIG IDEA 3 (Pt 1): VARIABLES · CONDITIONALS · LOOPS (30–35%)

This is the **biggest unit on the exam**. Master these constructs in AP pseudocode — many MCQs are "trace this code, what is the output?"

Variables & Assignment [3]

```
count ← 0           // assigns 0 to count
count ← count + 1    // increments by 1 → count is now 1
name ← "Alice"       // strings use quotes
flag ← TRUE          // booleans are TRUE / FALSE
```

Conditionals — IF / ELSE [3]/[4]

```
IF (score >= 90) {
    grade ← "A"
}
ELSE IF (score >= 80) {
    grade ← "B"
}
ELSE {
    grade ← "C or below"
}
```

Boolean operators [3]/[4]: AND (both true), OR (either true), NOT (flips). **Short-circuit [4]:** in (A AND B), if A is false, B isn't even evaluated.

Loops — REPEAT [3]/[4]

```
// REPEAT N TIMES — runs exactly N times
REPEAT 5 TIMES {
    DISPLAY("hi")      // prints "hi" 5 times
}

// REPEAT UNTIL — runs UNTIL condition is true
i ← 1
REPEAT UNTIL (i > 10) {
    DISPLAY(i)         // prints 1, 2, 3, ... 10
    i ← i + 1
}

// FOR EACH — runs once per item in a list
FOR EACH x IN [3, 7, 11] {
    DISPLAY(x * 2)     // prints 6, 14, 22
}
```

WATCH OUT [4] Off-by-one errors are everywhere. "REPEAT UNTIL i > 10" runs while i ≤ 10, so 10 IS printed. Hand-trace the first and last iterations.

Lists [3]/[4] — 1-INDEXED!

```
scores ← [85, 92, 78, 95, 88] // length 5
scores[1] → 85 // FIRST item, not second!
scores[5] → 88 // LAST item
LENGTH(scores) → 5

APPEND(scores, 100) // scores is now [85,92,78,95,88,100]
INSERT(scores, 1, 50) // inserts 50 at position 1
// scores is now [50,85,92,78,95,88,100]
REMOVE(scores, 3) // removes item at position 3
```

PAGE 5 | BIG IDEA 3 (Pt 2): PROCEDURES · ALGORITHMS · BINARY SEARCH

Procedures (Functions) [3]/[4]

```

PROCEDURE max(a, b) {
  IF (a > b) { RETURN(a) }
  ELSE      { RETURN(b) }
}

bigger ← max(7, 12)           // bigger is 12

// Parameters = inputs (a, b above)
// Return value = the output
// CALLER: uses the procedure.  IMPLEMENTATION: defines it.

```

KEY INSIGHT [4] Procedural abstraction is a key Create PT row-point: defining your OWN procedure with parameters that does meaningful work. Examples: a procedure that takes a list and returns its average; one that takes (length, width) and returns the area.

Algorithms — Linear vs Binary Search [3]/[4]/[5]

Algorithm	How it works	Speed (n items)
Linear search [3]	Check each item one-by-one until match found.	Worst case: n steps (must check all).
Binary search [4]	Repeatedly halve the range; check the middle. Requires SORTED data.	Worst case: $\log_2(n)$ steps — much faster.
LINEAR SEARCH 1,000,000 items → up to 1,000,000 checks BINARY SEARCH 1,000,000 items → only ~20 checks ($\log_2(1,000,000) \approx 20$) Doubling the data: Linear: doubles the time Binary: adds just ONE more step		
WATCH OUT [4] Binary search trap. If a question shows UNSORTED data, binary search doesn't work — only linear. Always check whether data is sorted before claiming binary search applies.		

Algorithm Efficiency [4]/[5]

- **[4] Reasonable time** = polynomial growth (n , n^2 , n^3). **Unreasonable** = exponential growth (2^n) or factorial ($n!$) — too slow for big n .
- **[5] Decision problem** = a problem with a yes/no answer. **Optimization problem** = find the best solution among many (e.g., shortest route).
- **[5] Heuristic** = an approximate algorithm that finds a "good enough" solution when the exact answer is unreasonably slow. Used for traveling salesman, complex optimization.
- **[5] Undecidable problem** = no algorithm can solve it for ALL inputs (e.g., Halting Problem).

Random & Simulation [4]

```

RANDOM(1, 6)    // returns random integer between 1 and 6 inclusive
// Used for: simulations, games, sampling, AI training data

```

PAGE 6 | BIG IDEA 4: COMPUTER SYSTEMS & NETWORKS (11–15%)

How the Internet Works [3]/[4]

- **The Internet [3]** = a network of networks using common protocols (TCP/IP). NO central authority. Open standard.
- **Web [3]** ≠ Internet. The **Web** is one application built on top (HTTP, HTML pages). Email, video calls, gaming use the Internet but aren't "the Web".
- **Packet switching [3]/[4]** = data is split into small **packets**, each routed independently, reassembled at destination.
- **Why packets, not whole files? [4]** Resilience — if one path fails, packets reroute. Better bandwidth sharing. No single connection hogs the line.

Protocols & Layers [4]/[5]

Protocol/Layer	What it does
IP (Internet Protocol)	Addresses + routes packets. Each device has an IP address.
TCP (Transmission Control Protocol)	Ensures packets arrive in order + reliably. Reorders, requests resends. Slower but reliable.
UDP [5]	Faster, no resends. Used for streaming/games where speed > perfection.
HTTP / HTTPS	Web pages. HTTPS is encrypted (uses TLS/SSL).
DNS	Translates domain names (google.com) → IP addresses.
[5] Net neutrality	Principle that ISPs treat all traffic equally — not a protocol, a policy debate.

Fault Tolerance & Redundancy [3]/[4]

- **Fault tolerance [3]** = system keeps working even when components fail.
- **Redundancy [3]** = duplicate paths/components so if one fails, another takes over. The Internet is fault-tolerant BECAUSE it has redundant paths between nodes.

KEY INSIGHT [3] Frequent MCQ. "Why is the Internet fault-tolerant?" → Because it has **redundant routing paths**. Packets can take different routes if one fails.

Parallel & Distributed Computing [4]/[5]

- **Sequential [3]** = one step at a time.
- **Parallel [4]** = multiple parts of one task running simultaneously on **multiple processors of one machine**. Limited by the slowest part + sequential portion (Amdahl's Law conceptually).
- **Distributed [4]** = task runs on **multiple separate machines** connected by a network. Used for huge problems: protein folding, weather, search indexing.
- **[5] Speedup** = sequential time ÷ parallel time. Cannot exceed the number of processors. Cannot exceed (1/sequential portion) — there's always a limit.

PAGE 7 | BIG IDEA 5: IMPACT OF COMPUTING (21–26% — DON'T SKIP!)

This is 21-26% of the exam — **bigger than Big Ideas 1, 2, or 4**. Easy points if you read carefully and spot the right answer choice. Most questions test whether you recognize fairness, accessibility, privacy, and security concerns.

The Digital Divide [3]/[4]

- **Digital divide [3]** = unequal access to computing/internet, often along lines of income, geography (rural vs urban), education, age, country.
- **Why it matters [4]**: people without access miss out on jobs, education, government services, healthcare. Software designed without considering low-bandwidth or older devices excludes them.

Bias in Computing [4]/[5]

- **Algorithmic bias [4]** = software produces unfair outcomes for some groups, often because training DATA is biased.
- **Examples [4]/[5]**: facial recognition less accurate on darker skin (training data was mostly light-skinned); resume screening preferring male names; loan algorithms denying minorities.
- **[5] Mitigation**: diverse development teams, inclusive testing, audit datasets for representation, transparency about how decisions are made.

Crowdsourcing & Citizen Science [4]

- **Crowdsourcing [4]** = getting contributions from large online groups. Wikipedia, OpenStreetMap, GoFundMe, citizen science projects.
- **Benefits [4]**: scale, diversity, low cost. **Risks [4]**: quality control, vandalism, bias toward who can contribute.

Intellectual Property [3]/[4]

Concept	What it means
Copyright [3]	Legal right of creator to control use of their work. Default — no permission needed to GET, but can't copy/distribute.
Creative Commons [3]/[4]	Licenses that grant SOME rights (e.g., "you can use if you credit me"). Many flavors: CC-BY, CC-BY-SA, CC-BY-NC.
Open source [4]	Source code is freely available — others can read, modify, redistribute (subject to license).
Public domain [4]	No copyright protection — free for anyone to use any way (old works, government works).
Plagiarism [3]	Using someone else's work without credit. ALWAYS cite borrowed code in your Create PT.

Privacy & Security [3]/[4]/[5]

- **PII [3]** (Personally Identifiable Information) = name, SSN, address, biometrics. **Don't collect/share PII unnecessarily.**
- **Encryption [3]/[4]** = scrambling data so only authorized parties can read it. Symmetric (one shared key) vs **public-key/asymmetric [5]** (public key encrypts, only private key decrypts).
- **Common attacks [4]/[5]**: **phishing** (fake emails to steal credentials), **malware** (malicious software), **rogue access point**, **DDoS** (flood a server), **keylogger** (records keystrokes).
- **Multi-factor authentication (MFA) [4]** = something you KNOW (password) + something you HAVE (phone/code) + something you ARE (biometric). Defense in depth.
- **[5] Cookies, browsing history, location data** = digital footprints. Even "anonymized" data can often be re-identified by combining with other datasets.

PAGE 8 | THE CREATE PERFORMANCE TASK — 6 ROW-POINT CHECKLIST

You complete the Create PT in class **BEFORE** exam day. It's submitted to College Board. The PT counts for ~25% of your final score, graded on **6 row-points (0 or 1 each)**. Each row is a simple yes/no — does your code/writeup contain X? Hit all 6 = perfect score. Below is the universal checklist.

THE 6 ROW-POINTS — must-hit for any tier

Row	What it tests	Requirement	Tier needed
1	Program purpose & function	Writeup describes purpose AND a function the program performs (clearly).	[3]
2	Function development	Code includes a STUDENT-DEVELOPED procedure that does meaningful work (not just calls a built-in).	[3]
3	Use of a list	Code uses a LIST (or similar collection — array, dictionary) to manage complexity.	[3]/[4]
4	Use of iteration	Code includes a LOOP that contributes to fulfilling program purpose.	[3]/[4]
5	Procedure with parameters	Procedure (Row 2) has ≥1 parameter that meaningfully affects behavior.	[4]
6	Algorithm w/ selection AND iteration	An algorithm INSIDE that procedure uses BOTH a conditional AND a loop.	[4]/[5]

KEY INSIGHT The simplest design that hits all 6 row-points: a procedure that takes a list and a target value as parameters; uses a loop + an IF to count occurrences (or filter); returns the result. This satisfies Rows 2, 3, 4, 5, AND 6 in one move.

CREATE PT — common deductions [4]/[5]

- [4] Procedure does "trivial" work — just adds two numbers, just prints. Make it iterate or filter.
- [4] Parameter doesn't actually matter — passing a parameter that isn't used inside the procedure body fails Row 5.
- [4] List is decorative — must be USED (looped, modified, queried), not just declared.
- [5] Writeup describes wrong code — match the writeup language to actual code variable/procedure names.
- [5] Code from someone else uncited — must explicitly state "this code/idea was from X" if you used a tutorial. Plagiarism = automatic 0.

CREATE PT — submission requirements [3]

- Video [3] (≤1 min, no narration of yourself, just screen capture) showing input → program running → output.
- Personalized Project Reference (PPR) [3] — PDF showing screenshots of: your procedure (Row 2), where it's CALLED, your list (Row 3), and your iteration (Row 4).
- Written Response [3] — short answers about purpose, the procedure, the list, your testing. Match the variable names exactly.

PAGE 9 | EXAM STRATEGY · MCQ TYPES · TRACING CODE

MCQ Question Types — what to expect [3]/[4]

Type	Strategy
Trace this code, what's the output?	Hand-trace with a table: write each variable name, update on every line. Don't try to run it in your head — write it out.
Which code segment does X?	Read each option carefully. Test against an EDGE case (empty list, n=1, all-same).
Which scenario is best for X technology/algorithm?	Match the FEATURE (sorted? large? real-time?) to the algorithm strength.
Which is true about [Big Idea 5 topic]?	Pick the answer mentioning fairness/accessibility/privacy. Avoid extreme/absolute language.
Two-part questions (last 5 q's)	Each part has separate points. Don't skip — even a partial guess can earn a point.

THE TRACE TABLE — your superpower [3]/[4]

Every "what's the output?" question becomes easy if you make a table. Example: trace this code with input list [4, 7, 2, 9, 5]:

```

PROCEDURE mystery(aList) {
  result ← 0
  FOR EACH x IN aList {
    IF (x > result) { result ← x }
  }
  RETURN(result)
}

```

Trace table (write this on scratch paper):

x (current)	result	Notes
—	0	Initial
4	4	4 > 0 → update
7	7	7 > 4 → update
2	7	2 > 7? No → no change
9	9	9 > 7 → update
5	9	5 > 9? No → no change
END	9	Procedure returns 9 — finds the MAX

MCQ TIME STRATEGY [3]/[4]/[5]

- **70 questions in 120 minutes** = ~1:42 per question. Most take 30-60 sec; trace questions take 2-3 min.
- **[3] Pass strategy:** answer all easy/Big-Idea-5 questions first (fast). Skip long traces for the second pass.
- **[4] Strategy:** do questions in order BUT flag any trace > 90 sec — return at the end if time.
- **[5] Strategy:** aim for ~80 sec on early questions to bank time for hard traces. Read EVERY option (College Board often has 2 "close" answers — pick the more precise one).
- **Answer EVERY question** — no penalty for wrong. Random guess = 25% chance, that's free expected value.

PAGE 10 | SCORE LADDER · TIER CHECKLISTS · THE ONE THING

THE SCORE LADDER — your three blueprints

For a 3 (Pass)	For a 4	For a 5
MCQ: 33-38 / 70	MCQ: 42-46 / 70	MCQ: 52-58 / 70
Pseudocode literacy: trace simple loops, lists.	Trace nested loops, parameters in procedures.	Trace tricky boundary cases, recursion-like patterns.
Big Idea 5: spot fairness/privacy answers.	Big Idea 5: distinguish copyright vs CC vs public domain.	Big Idea 5: nuanced cybersecurity (MFA, encryption types).
Networks: redundancy, packet switching, fault tolerance.	Networks: TCP vs UDP, parallel vs distributed.	Networks: speedup limits, net neutrality, IPv4 vs IPv6 reasoning.
PT: hits Rows 1, 2, 3, 4 (4/6).	PT: hits Rows 1-5 (5/6).	PT: hits all 6 rows perfectly.
Compression: lossless vs lossy basics.	Compression: when to use which, irreversibility.	Algorithm efficiency, undecidable, heuristic.

TIER-SPECIFIC TEST-DAY CHECKLISTS

For a 3 — focus only on these:

- ✓ MCQ: 33+ correct.
- ✓ Hand-trace simple loops on scratch paper. Lists are 1-INDEXED.
- ✓ Big Idea 5 questions — pick the answer mentioning fairness, accessibility, privacy.
- ✓ Internet basics: packet switching + redundancy = fault-tolerant.
- ✓ PT submitted with: a procedure, a list, a loop, a video, a writeup.

For a 4 — add these on top:

- ✓ MCQ: 42+ correct.
- ✓ Procedure has meaningful PARAMETER that affects behavior.
- ✓ Inside that procedure: BOTH a conditional AND a loop.
- ✓ Distinguish lossy vs lossless; binary search needs SORTED data.
- ✓ TCP vs UDP, parallel vs distributed.
- ✓ Algorithmic bias = bad training data → unfair outputs.

For a 5 — add these on top:

- ✓ MCQ: 52+ correct.
- ✓ All 6 PT row-points perfect; writeup matches code variable names exactly.
- ✓ Reasonable vs unreasonable time (polynomial vs exponential).
- ✓ Heuristic, undecidable, decision problem distinctions.
- ✓ Public-key encryption: public encrypts, private decrypts.
- ✓ Speedup limits in parallel computing — sequential portion caps gain.
- ✓ Re-identification of "anonymized" data via dataset combination.

THE ONE THING THAT MATTERS MOST [3][4][5]: HAND-TRACE EVERY CODE QUESTION ON SCRATCH PAPER. AP CSP MCQs reward the patient student. The students who fail aren't the ones who don't know — they're the ones who try to run code in their heads and lose track of variables. Make a trace table, update one line at a time, and you'll get questions right that look impossible at first glance. Combined with hitting all 6 PT rows, this is the formula.

If you do nothing else: nail the 6 PT rows BEFORE exam day, and trace MCQ code on paper. That's how a 3 becomes a 5 in CSP.

DISCLOSURE & TERMS OF USE

This study guide is published by **Wayfinder** (wayfinderai.org) for educational and self-study purposes only.

Not affiliated with the College Board

AP®, Advanced Placement®, and the names of all individual AP exams are registered trademarks of the College Board, which neither sponsors nor endorses this guide. Wayfinder is an independent educational publisher with no affiliation, partnership, or formal relationship with the College Board, ETS, or any other testing organization. Any references to the College Board, the AP Program, exam frameworks, scoring rubrics, or Course and Exam Descriptions are made for educational commentary, criticism, and study purposes consistent with fair use.

No guarantee of results

Wayfinder makes no representation, warranty, or guarantee — express or implied — that use of this guide will produce any particular AP exam score, college admission outcome, course grade, or academic result. AP exam performance depends on many factors outside our control, including the student's preparation history, individual aptitude, exam-day conditions, scoring rubric changes, and the work each student puts in. The strategies, frameworks, sample responses, and practice content in this guide reflect our best understanding of the relevant AP exam at time of publication; the College Board updates its exams and scoring guidelines periodically and may have done so since this guide was written.

Educational use only

The content in this guide is provided "as is" for the personal educational use of the original purchaser. It is not a substitute for the student's own classroom instruction, official College Board materials (the Course and Exam Description, released free-response questions, or AP Classroom), guidance from a qualified

teacher, or independent verification of facts. We strongly encourage students to use this guide alongside — not in place of — official AP resources and their own school's curriculum.

Sample responses

Any sample student responses, model essays, exemplar paragraphs, or scored sample answers in this guide are illustrative compositions written by Wayfinder for instructional purposes. They are not actual student work, are not actual College Board materials, and have not been scored by official AP readers. They demonstrate writing patterns and rubric-aligned reasoning rather than guaranteed-score work.

Intellectual property

This guide is the copyrighted work of Wayfinder. You may print one copy for your personal study use. You may not redistribute, resell, repost, share via file-sharing services, upload to study-help platforms (e.g., Course Hero, Chegg, Quizlet), or use any portion to train AI/ML models without our express written permission. Quotations of brief passages for review or commentary purposes are permitted with attribution.

Limitation of liability

To the maximum extent permitted by law, Wayfinder, its affiliates, contributors, and licensors disclaim all liability for any direct, indirect, incidental, consequential, special, or exemplary damages arising from or related to the use of this guide, including but not limited to errors or omissions in content, reliance on study strategies, score outcomes, missed admission deadlines, or any other educational or financial result. This guide is provided without warranty of any kind, whether express or implied, including without limitation any implied warranty of merchantability, fitness for a particular purpose, or non-infringement.

Errors and updates

We aim for accuracy but errors are possible. If you find a mistake — a misstated fact, a stale rubric reference, a broken example — please email [**support@wayfinderai.org**](mailto:support@wayfinderai.org) so we can correct it in the next revision. Use of this guide constitutes acknowledgment that exam content may have changed since publication and that you are responsible for cross-checking critical information against the most current official AP source.

Governing terms

Use of this guide is governed by Wayfinder's full Terms of Service and Privacy Policy, available at wayfinderai.org/terms.html and wayfinderai.org/privacy.html. By using this guide you agree to those terms.

© Wayfinder. All rights reserved.